Attestor Technical Architecture White Paper

Version: v0.3.0-evaluation

Scope: Repository-side architecture, local multi-path evaluation, integration model, and proof contracts.

TABLE OF CONTENTS

Part I — Core Architecture

I. Problem, Engine and System Boundary

1. Abstract — From AI Intent to Controlled Consequence

The capabilities of AI systems extend beyond text generation; they prepare operations that carry real business consequences. An AI is capable of calculating a refund, exporting data, modifying permissions, or provisioning infrastructure.

The problem is that the output generated by the model alone cannot authenticate the authorization, safety, and intent of the execution. An independent control layer is required. The system is a consequence admission control plane that structures the proposed operation and decides, within a strict, deterministic framework, whether it is sufficiently justified to be forwarded to the downstream system.

2. AI Intent Is Not Execution Authority

Classic "agentic" models often incorrectly couple intent directly with execution, assuming the model output can simply call a tool and produce a consequence.

This approach is dangerous. The output of the model alone fails to prove the following:

- **Authority:** Does the operation possess the appropriate authorization?
- **Evidence:** Is there independent evidence supporting the claims?
- **Approval:** Has the necessary human or delegated approval occurred?
- **Scope & Freshness:** Is the scope of the operation safe, and has the approval expired?
- **Replay Safety:** Is this a prohibited repetition of a prior operation?

Uncertainty must be strictly blocked from becoming execution authority.

3. What Is Attestor?

The system serves exclusively as an intermediary, independent control layer (control plane) that sits between the intent and the customer's execution environment.

4. One Nested, Interlocking Consequence Control Engine

The platform operates as a single, nested, interlocking consequence control engine built intentionally around the AI system.

Each layer is embedded in the same consequence lifecycle. It receives the same bounded consequence identity and extends it with a new form of verified state: action-surface context, evidence provenance, authority posture, policy scope, risk constraints, admission outcome, release binding, execution result, and proof lineage.

The output of one layer becomes a verified part of the same consequence record and constrains what every later layer is allowed to do.

At the centre of the engine is one continuous question:

May this exact consequence cross into execution under the current policy, authority, evidence, scope, freshness, and enforcement conditions?

The runtime answers that question for a concrete request. Shadow observation and Policy Foundry improve the evidence and policy available to future requests. Policy governance controls which rules are active. Cryptographic artifacts bind the decision and authorization to verifiable integrity material. The customer enforcement point applies that authorization at the real execution boundary. Receipts, outcomes, and incidents close the same control loop.

These layers operate at different moments as interlocking parts of one unified control system, sharing one consequence identity, one authority boundary, one policy context, and one proof lineage.

5. The Four Control Planes

The architecture strictly separates the different control planes to make it clear what can authorize execution:

- **Decision Authority Plane:** Only hard gates, approved policies, and valid review paths can grant execution permission.
- **Signal Relationship Plane:** Governs how signals may confirm, conflict, depend on, or constrain one another without creating execution authority.
- **Audit Plane:** Manages evidence of decision, release, verification, and receipt events.
- **Measurement Plane:** Monitors metrics, incidents, drift, and degraded conditions while remaining strictly outside the execution authorization path.

Figure 1. The Core Architecture Model

Threat Model and Non-Goals

The system operates under a specific threat model and explicitly bounds what it protects:

- **Adversaries:** The system assumes the AI is compromised or hostile. Attackers may include prompt injection, poisoned tool output, replay attackers, or a compromised agent.
- **Customer Integration:** Attestor does not protect a downstream system that the customer has left bypassable. Without a strict customer PEP, protection is incomplete.
- **Identity and Custody:** Attestor does not replace standard IAM, wallet custodians, KMS providers, or customer policy ownership. It integrates with them.
- **Release Authorization Boundary:** A release authorization carries bounded permission for one consequence; it is not a reusable bearer permit.

  At the downstream execution boundary, the Customer PEP independently verifies that the presented authorization matches the actual request, intended audience, expiry window, replay state, and permitted scope before allowing execution.

II. Understanding the Customer Action Surface

6. Action-Surface Discovery

Discovery maps the unstructured external world into a strictly bounded action-surface model.

The process begins by analyzing raw protocol schemas (such as OpenAPI definitions or RPC descriptors), parsing their paths, parameters, and descriptions to identify the target consequence domain. Instead of treating every endpoint equally, it assigns a specific taxonomy domain—like `money-movement` or `data-disclosure`—to each recognized action.

This yields a set of Action Surface Declarations that tell the engine exactly what it is protecting, what dimensions are relevant for policy, and which fields must be sanitized before any live traffic is ever evaluated.

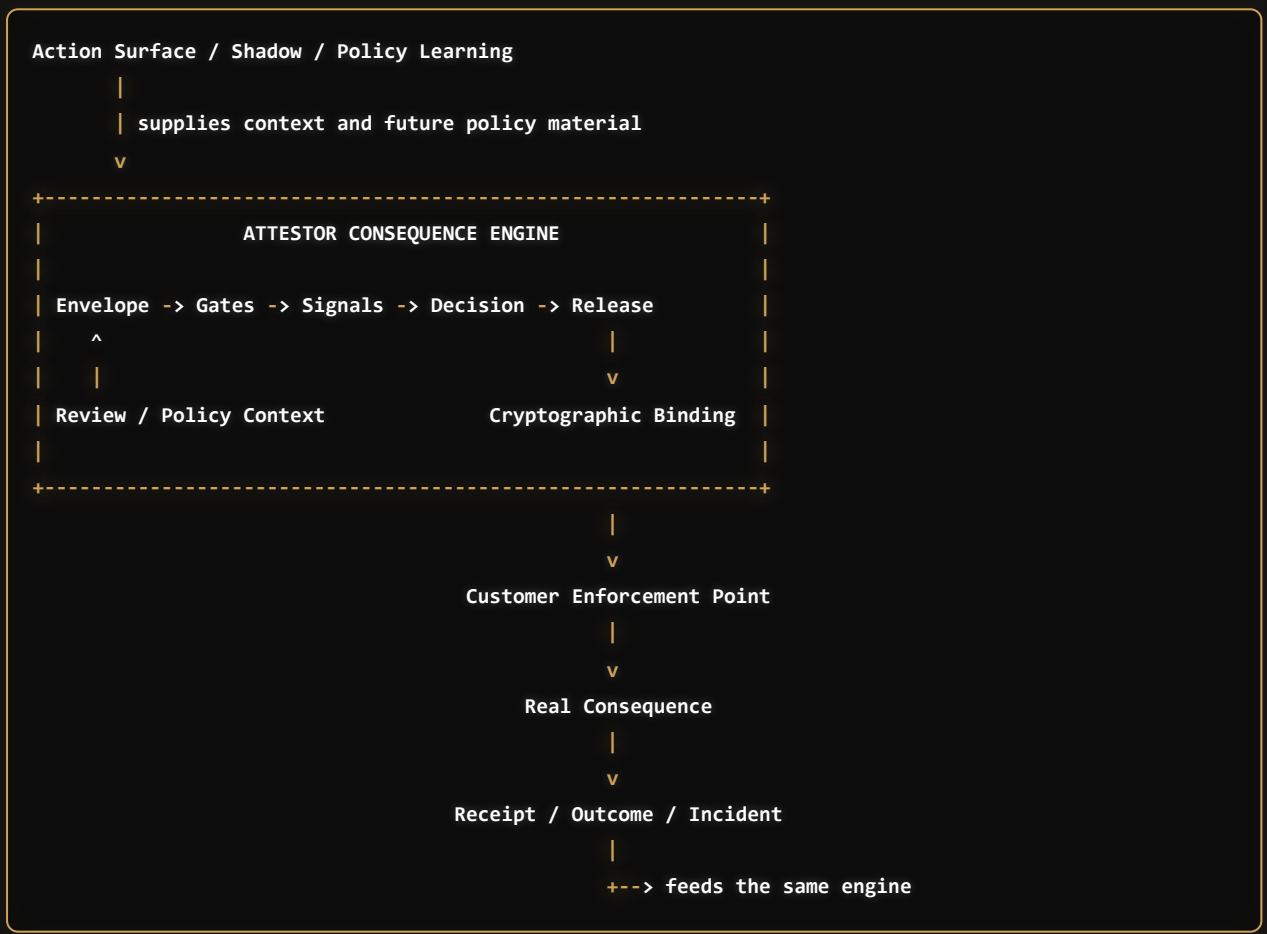
7. Canonical Shadow Events

Shadow observation safely builds operational evidence without exposing sensitive raw material to the measurement or policy planes.

As the system processes live payloads, it maps them against the action-surface model to extract only the necessary semantic facts required for policy evaluation. During this mapping, it explicitly strips out raw AI prompts, underlying provider payloads, and private customer identifiers to maintain strict data minimization.

The result is a stream of Canonical Shadow Events. These strictly typed, digest-bound artifacts form a safe, privacy-preserving foundation for future policy discovery and simulation without leaking execution details.

Figure 2. The platform is one nested consequence control engine.



8. Evidence State, Provenance and Trust Classification

Evidence classification ensures that the runtime only acts on verifiable, cryptographically sound information.

When claims enter the system—such as a user approval or a context signal—the classification layer evaluates their age, source identity, and cryptographic integrity against established trust criteria. If a source is expired, unsigned, or unverified, the engine does not silently fail or guess the intent; it actively downgrades the evidence into a blocking state.

This strict evaluation yields a structured evidence state for the consequence. By explicitly classifying provenance, the system guarantees that unverified claims or hallucinated model outputs can never masquerade as legitimate execution authority.

9. Integration Planning and No-Bypass Design

Integration planning explicitly defines where the customer-controlled enforcement boundary must be placed to prevent downstream circumvention.

By analyzing the declared action-surface model, the system generates testable attack vectors that simulate bypassed controls. These vectors are designed to execute successfully against an unprotected endpoint, but fail against a properly configured Customer Enforcement Point (PEP).

This process generates an Integration Kit that maps the deployment to expected live-proof evidence. The protection loop remains fundamentally incomplete until the customer deployment provides evidence that the configured boundary successfully blocked these defined bypass attempts.

10. Adoption and Evaluation Postures

The system strictly evaluates integration readiness.

Before transitioning a tenant to active enforcement, the control plane evaluates the customer's integration architecture and credential management. It scans for dangerous anti-patterns, such as static downstream API secrets being held directly by the AI agent rather than the PEP.

This evaluation outputs a deployment posture. Active enforcement is permitted only when the posture satisfies the required credential-isolation and boundary-readiness conditions; otherwise, the engine remains in an observe-only evaluation mode.

Part II — Runtime, Policy, Proof and Enforcement

III. Consequence Runtime

11. Canonical Action Taxonomy and Intent Normalization

Before evaluation begins, the first control point is the intent normalizer. The messy AI or workflow output is transformed into a Canonical Consequence Envelope.

All discovered actions are mapped to a canonical, typed consequence domain, such as `money-movement`, `data-disclosure`, `authority-change`, or `programmable-money`. The system does not write custom policy logic per API endpoint. Instead, it classifies the intent into one of these strict taxonomy domains.

This mapping is not just for routing; it immediately subjects the action to a hardcoded enforcement matrix. Every domain in the taxonomy defines a minimum Risk Class (e.g., R3 or R4) and a mandatory set of Control Requirements. For example, any action mapped to `money-movement` or `authority-change` must structurally prove `evidence-binding`, `replay-protection`, `downstream-verification`, and a `human-review-path`. If an incoming AI intent is classified under a high-risk taxonomy but fails to provide the required controls, the admission engine structurally rejects it.

If the input is invalid, incomplete, or unbound, the system operates in a "fail-closed" manner. An unbound input results in a hold, meaning no release and no downstream action.

The Envelope itself precisely records the actor, action, target, tenant, downstream system, as well as evidence and policy scope references. The Canonical Consequence Envelope functions as the shared control object carried through the runtime. Every later decision, policy binding, review result, release authorization, and execution receipt refers back to this same bounded consequence.

12. Admission State Machine

The incoming consequence proceeds through a strict state machine from proposal through tracing to a decision, and finally enforcement. This flow is managed by the core generic admission engine, which evaluates the consequence against over a dozen specialized guards, including Authority Creep, Scope Explosion, Tool Result Poisoning, Agentic Supply Chain Risks, and the No-Go Condition Ledger.

Rather than making an immediate pass/fail choice, the state machine first calculates a `shadowDecision` (`would_block`, `would_review`, `would_narrow`, or `would_admit`). This purely mathematical observation is then translated into an `effectiveDecision` and a `downstreamPosture` depending on the active evaluation mode. For instance, if the engine is in `observe` mode, a `would_block` shadow decision is recorded, but the effective posture remains `observe-only`. If the engine is in `enforce` mode, the same shadow decision translates directly into a hard `block` effective decision.

Main invariants prevent dangerous state transitions at every step. If there is no authority, the Authority Guard forces a block. If a downstream secret is exposed, the No-Go Ledger forces a block. A required review path cannot be bypassed. A consumed nonce guarantees no second execution. This rigorous, multi-stage evaluation ensures that execution authority is never granted by accident or default, but only through explicit, verified progression through every required control. The admission flow is also modeled by a separate TLA+ design state machine, which records the main safety invariants guiding the TypeScript runtime.

13. Hard Admission Gates

The Hard Gates phase blocks inherently impossible execution attempts before they reach complex fusion.

The system evaluates the incoming Canonical Consequence Envelope against specific, digest-bound constraints. For example, the Scope Explosion Guard compares the requested intent dimensions against the approved policy boundaries.

If an action exceeds its boundary, a narrow outcome is permitted only when policy can define a smaller, fully bounded consequence with a distinct target, scope, evidence binding, and downstream presentation. Otherwise, it emits a hard block. This deterministic evaluation outcome is then passed to the Fusion layer for further semantic analysis.

14. Agent Identity, Delegation and Tool Boundaries

When multiple agents collaborate, the system strictly traces the identity chain using the Multi-Agent Delegation Guard. Every entity in the chain must provide a cryptographically digested identity, a defined principal kind (`human-user`, `ai-agent`, `tool`, `workflow`), and an explicit role (`originator`, `delegator`, `delegate`, `executor`, `approver`, `auditor`).

The system does not rely on trust between collaborating agents; it mechanically enforces boundaries across the entire chain. For instance, the engine explicitly extracts the digests of all `approver` roles and compares them against the `executor` roles. If an intersection is found, the system immediately blocks the action for `delegation-actor-self-approved`, structurally preventing an AI from both proposing and authorizing its own execution.

Furthermore, the delegation guard prevents structural collapse by evaluating the chain's topology. It detects cyclic delegations by matching identity digests, enforces a strict maximum delegation depth, and immediately halts cross-tenant calls unless an explicit, cross-tenant approved scope digest is provided. This ensures that even in complex, multi-agent swarms or extended tool supply chains, accountability remains unbroken, mathematically traceable, and constrained to the originally delegated scope.

15. Typed Signals and Layer Opinions

The Measurement Plane standardizes how different evaluators express findings.

Different evaluators may contribute typed opinions about hazards, gaps, boundaries, context, or measurement. Each opinion is bound to the same consequence record and carries its source, scope, evidence references, and limits.

An opinion may expose risk, uncertainty, or missing evidence, but it cannot create authority, lower a required review level, or declare a consequence safe.

16. Signal Relationship Fabric

The Signal Relationship Fabric is the typed semantic layer that determines how signals are allowed to interact inside one consequence record.

It is not a scoring table and it is not a second decision engine. Its purpose is to preserve the meaning, provenance, authority level, and dependency of each signal before any fusion or final decision takes place. Every signal enters the Fabric with a defined category, source, evidence binding, authority posture, scope, freshness, and confidence boundary. The Fabric then records how that signal relates to other signals affecting the same consequence.

Relationships are directional where authority matters and symmetric where the relationship describes a shared condition. The Fabric determines if signals confirm the same hazard, contradict one another, duplicate correlated evidence, or depend on missing input. It permits higher-authority evidence to override advisory signals, and allows context to modulate severity or escalate caution. It can suppress a lower-trust signal, or explicitly force the consequence into a human review path.

The Fabric enforces authority-preserving rules. An advisory signal cannot create execution authority. A measurement signal cannot weaken a hard admission condition. Duplicate evidence cannot amplify risk by being counted repeatedly. Suppression cannot erase a hard floor, formal denial, or established authority failure. Higher-trust evidence may constrain interpretation, but it cannot silently convert uncertainty into permission.

The Fabric therefore produces a governed interaction graph rather than a flat list of signals. Relationship-Aware Monotone Fusion evaluates that graph in the next stage to derive a bounded caution posture for the same consequence.

17. Context Modulators

Context modulators define environmental factors that influence the decision engine, operating strictly on dimensions such as reversibility, blast radius, tenant maturity, coverage, and freshness. They act as dynamic context parameters within the authority boundary.

Context modulators interpret otherwise valid signals in light of reversibility, blast radius, tenant posture, coverage, and freshness. They may increase evidence requirements, review pressure, block pressure, or narrowing requirements.

They cannot create authority, weaken a hard denial, or convert a lower-risk context into permission. They exclusively determine how strictly the engine interprets otherwise valid signals, ensuring that high-risk contexts dynamically elevate security requirements without ever creating a loophole to lower them.

18. Relationship-Aware Monotone Fusion

Relationship-Aware Monotone Fusion is not a confidence-scoring model and it is not a second authority engine. Its role is to evaluate how independently produced signals interact while preserving the authority boundary already established by the hard admission gates.

The engine does not treat all signals as equal or independent. It takes the typed signals, evidence provenance, source dependencies, relationship rules, and context modulators, and derives a fusion posture.

The resulting posture may be clear, watch, review, or block-pressure. A posture is not itself an execution decision. 'Clear' does not independently create permission to execute. 'Watch' preserves the consequence while recording elevated observation. 'Review' requires the consequence to enter a bounded human decision path. 'Block-pressure' raises the strictest caution posture before final decision synthesis.

The monotone property protects the hard boundary. A new independent risk may increase caution, and new uncertainty may require review or a hold. Duplicate evidence must not be counted twice. Higher-trust evidence may suppress lower-trust advisory relevance. However, a hard authority failure cannot be suppressed, averaged away, or overridden by fusion. Fusion therefore does not simply make the system stricter; it prevents the engine from becoming falsely permissive when evidence is duplicated, contradictory, incomplete, or derived from a lower-trust source.

19. Conflict, Abstention and Human Comprehension

When Fusion identifies risk or uncertainty, the engine must determine whether the conflict can be safely resolved. A conflict may be resolvable if new, verifiable evidence is supplied, or it may require explicit human intervention. However, if the conflict involves a fundamental authority violation, tenant isolation breach, or a hard "no-go" hold, even human override is strictly forbidden.

Abstention is not the absence of a decision; it is an explicit refusal to infer permission from incomplete, contradictory, stale, or insufficiently independent evidence. If the engine must abstain but the operation is theoretically permissible, it shifts the consequence into a bounded review path.

When a review is needed, the system generates a structured review packet containing a short, focused, human-readable list of problems. The reviewer is presented with specific, constrained questions rather than raw, potentially misleading, or overloaded telemetry. The answer provided by the reviewer does not bypass the engine; instead, it becomes new scoped authority evidence that feeds back into the consequence record for fresh evaluation.

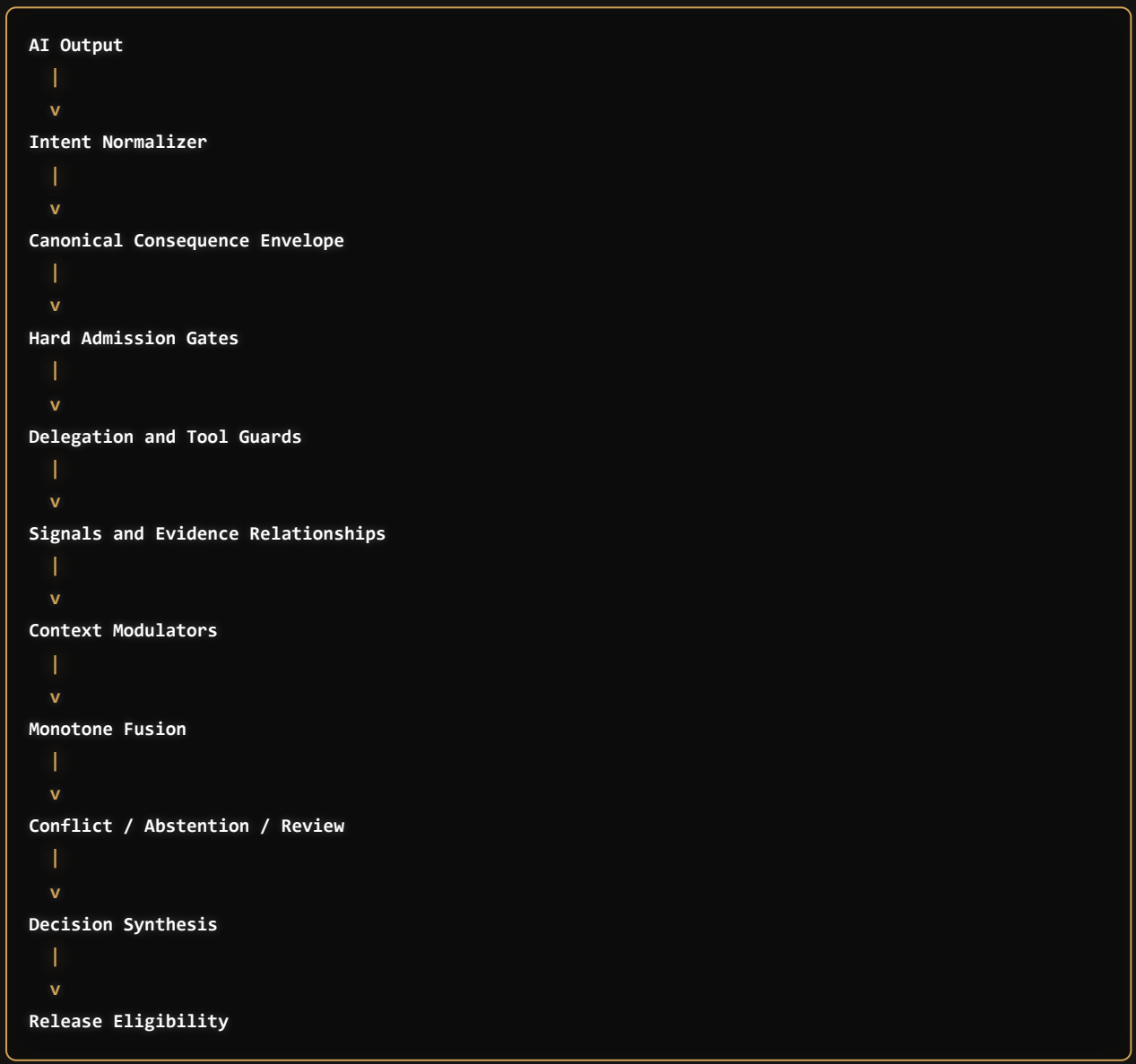
20. Decision Synthesis

Decision synthesis evaluates the final consequence posture to produce an actionable outcome.

The engine takes the fully evaluated state—including passed hard gates, the monotone fusion posture, and any human review results—and translates it into one of four distinct transitions. If all mandatory authority, policy, evidence, and risk conditions successfully close, it generates an Admit outcome. If the original consequence cannot proceed but policy allows a smaller, fully bounded action, it generates a Narrow outcome. If execution permission is missing and new scoped authority or evidence is needed, it defaults to a Review outcome. Finally, if a condition is violated where no safe transition is possible, it issues a Block outcome.

Decision synthesis does not create authority or invent evidence. It translates the fully evaluated consequence state into the next permitted lifecycle transition. An admit or narrow outcome creates release eligibility, not execution by itself. The next layer must turn that bounded decision into a verifiable authorization that remains tied to the same consequence record.

Figure 3. Inside the Consequence Runtime



IV. Review and Policy Governance

21. Review, Approval and Fresh Re-evaluation

A human review provides explicit authority, but it is intentionally designed so it never creates a runtime bypass or overrides core safety invariants.

When a policy mandates human intervention, the system does not simply forward raw telemetry or unstructured prompts to a dashboard. Instead, it extracts the exact consequence boundaries—such as the target entity, the requested financial amount, or the data scope—and presents the human with constrained, deterministic questions. The human action produces a scoped approval or rejection record bound to the consequence. Where a verified approval source or signed artifact is available, its provenance is evaluated as authority evidence during fresh admission.

Crucially, an approval does not blindly force the operation into execution. It simply injects this new authority evidence into the consequence record. Upon receiving this evidence, the system triggers a fresh admission evaluation. The engine must re-evaluate the current policy, tenant status, operational freshness, and drift conditions using the newly updated evidence state before it can transition the consequence into an eligible release state.

22. Decision Context Drift Binding

A previously approved decision, whether generated algorithmically or through explicit human review, remains usable only as long as its underlying context remains perfectly stable.

Every evaluated consequence is strictly bound to its decision-critical context through cryptographic digests. This context includes the target identity, the active policy version, the approved execution scope, relevant tool output material, and specific verification conditions. If any of these elements shift between the moment of approval and the moment of release, the system detects a state mismatch.

When that context materially differs at presentation time, the prior authorization immediately becomes invalid for the new state. This drift detection acts as a continuous safety net, automatically forcing the engine into a fresh evaluation cycle. By explicitly linking the authorization to its exact historical context, the system prevents stale approvals from being hijacked to authorize mutated or replay-attacked requests.

23. Policy Foundry

Figure 4. Policy Lifecycle

Policy evolution within the platform is strictly data-driven, relying on concrete evidence gathered from shadow observation and live runtime execution.

The Foundry operates as an offline analytical engine. It ingests the continuous stream of Canonical Shadow Events and recurring action patterns, translating them into structured candidate policy rules. Rather than deploying these rules immediately, the system executes extensive offline replays to test the candidates against historical consequence records and explicitly searches for dangerous counterexamples or unintended blocks.

This simulation and counterexample replay process provides rigorous mathematical and operational evidence about how a candidate policy would behave across both known traffic and constructed edge cases. While this reduces uncertainty and builds an empirical case for the rule, the Foundry itself cannot mutate the live runtime. Policy activation remains a strictly separated, human-governed timeline requiring explicit review before any new rule is enforced.

The Shadow observation provides input to the Foundry by mapping the action surface, but the Foundry itself never decides a live consequence. It operates offline.

Policy activation is a separate, human-governed timeline that ensures a generated rule only controls the runtime after strict review and approval.

24. Deterministic Policy Resolution

The engine cannot tolerate ambiguity when selecting the active ruleset; policy resolution must be completely deterministic.

When a consequence arrives, the runtime progressively narrows the applicable policy context. It filters available rules through the environment layer, the tenant and account boundaries, the specific action domain, the consequence type, the assessed risk class, and any active rollout scope. The engine traverses this hierarchy to locate the single most specific rule that governs the current operation.

If the resolution engine discovers multiple policies that are equally applicable, it does not attempt to guess or heuristically improvise a winner. Instead, it triggers a strict fail-closed mechanism. The system immediately outputs an `ambiguousEntryCandidates` error, mechanically blocking execution and returning a safe remediation path until human operators resolve the conflicting ruleset.

25. Roles, Separation of Duties and Dual Approval

To preserve the systemic integrity of the consequence engine, the Policy Control Plane enforces rigid boundaries regarding who may modify the active ruleset.

The platform structurally enforces Separation of Duties (SoD) across the entire policy lifecycle. By design, no single principal—whether an AI agent or a human administrator—may independently author, approve, and activate a high-impact policy change. The system explicitly isolates the roles that create candidate rules, conduct peer review, approve the simulation results, and finally activate the policy into the live environment.

This dual-approval structure guarantees that a compromised operator or a poisoned AI system cannot unilaterally relax the engine's boundaries to permit unauthorized actions.

26. Three Distinct Exception Mechanisms

While standard governance handles routine policy evolution, the system must accommodate operational emergencies without compromising its core safety invariants. It achieves this through three strictly segregated exception paths.

A **Review Approval** is used when a single consequence falls outside automated bounds but is operationally necessary. It translates human judgment into explicitly scoped authority evidence for that specific request alone, ensuring the broader governing policy remains completely untouched.

An **Execution Break-Glass** provides a time-bounded, explicitly reasoned override for hard operational limits. When an authorized operator pulls the break-glass, the system permits the execution to proceed but immutably binds permanent, high-severity warning findings to the consequence record, ensuring unavoidable post-incident review.

A **Policy Freeze or Rollback** acts as an emergency brake during sudden operational anomalies or suspected model drift. It halts the execution of an active policy or forcefully restores the system to a last-known-good state until shadow telemetry can be verified. Crucially, while these interventions provide operational flexibility, they can never bypass foundational `no-go` constraints —such as strict legal or cryptographic limits—which operate entirely below the exception layer.

V. Cryptographic Integrity, Proof References and Trust Artifacts

27. Canonicalization and Digest Binding

Within the consequence engine, trust-sensitive artifacts cannot be safely evaluated, signed, or transmitted as raw, floating JSON payloads.

When evaluation boundaries or release layers process a consequence, the system first transforms the payload into a strictly ordered, invariant byte representation. This canonicalization step strips away superficial formatting and nested differences, yielding a completely stable and deterministic format suitable for exact cryptographic integrity checks.

This transformation produces strict cryptographic digests that inextricably bind the governing policy, the collected evidence, the admission decision, and the release authorization to the exact same consequence identity. Any modification to the bound payload breaks digest verification. Replay attempts are handled separately through presentation binding, expiry, and single-use replay controls.

28. Proof References and Verification Material

Because a high-trust system cannot physically bundle gigabytes of historical telemetry into every live runtime execution, the engine relies on strict `ProofReference` artifacts to connect distributed evidence back to the original consequence record.

Rather than relying on brittle URLs or vague database IDs, the system constructs a highly structured pointer for every piece of critical evidence. A proof reference explicitly defines the artifact's semantic kind, its globally unique identifier, its canonical cryptographic digest, the URI where it can be retrieved, and specific programmatic verification guidance.

When the admission state machine or a downstream auditor evaluates a decision, this reference acts as an unbreakable link. It dictates exactly which external artifacts belong to the decision lineage and algorithmically defines how the downstream verifier must compute the payload hash to prove its integrity.

29. Signed Policy Bundles

To prevent rogue administrators or database tampering from silently altering the rules of the engine, the active policy itself must be mathematically verifiable at the exact moment of evaluation.

Instead of reading raw rows from a mutable database, the Policy Control Plane compiles the active, human-approved ruleset into a strictly canonicalized `SignedPolicyBundle`. This bundle contains the explicit domain routing, risk classifications, required review paths, and data-minimization rules, all sealed by a control-plane cryptographic signature.

During execution, the consequence engine requires this exact verifiable bundle as input. It hashes the bundle, checks the signature, and explicitly binds that specific policy digest to the consequence record. If the policy bundle is missing, its signature is invalid, or its digest differs from the expected tenant baseline, the admission state machine immediately and safely halts.

30. Signed Assurance Packets

While the runtime dictates *if* a consequence proceeds, the `SignedAssurancePacket` acts as the durable, offline-verifiable proof of *why* it was admitted.

When the Decision Synthesis layer reaches a terminal state (such as `admit` or `narrow`), the engine gathers the complete cryptographic history of the request. It packages the original Canonical Consequence Envelope digest, the active policy hashes, the ingested shadow signals, and the explicit human review timeline into a single, cohesive payload.

The engine then seals this payload into an integrity-verifiable artifact. By publishing this packet, the system allows third-party auditors, customer security teams, or downstream verifiers to completely reconstruct and validate the admission logic without requiring read access to the live control plane's internal database.

31. Signed Release Authorization

The `SignedReleaseAuthorization` is the specific, short-lived cryptographic token generated by the PAP that carries execution permission across the network boundary to the downstream PEP.

When the admission engine declares a consequence eligible for execution, it mints this token rather than calling the downstream API directly. The token embeds strict, immutable claims: the exact permitted target URL, the allowed action, the consequence digest, the expiration window, and the intended audience (the PEP identifier).

The signature on this authorization mathematically proves its origin and integrity. However, the Customer Enforcement Point (PEP) must still actively process the token. It structurally verifies the scope, audience, target, freshness, sender binding, replay state, and any required online status before allowing execution.

Release authorization strictly carries bounded execution permission from the decision layer to the enforcement boundary. It does not execute the consequence; it must still be presented and actively verified against the actual downstream request.

32. Trust Anchors and Key Custody

The integrity of the consequence engine depends on isolated key management and defined trust anchors.

When signing trust artifacts—whether they are policy bundles, assurance packets, or release authorizations—the system utilizes securely held private keys. The control plane tracks the nature of this custody, explicitly separating the keys used to sign governance policies from the keys used by the live runtime to mint release tokens.

These isolated trust anchors allow relying parties and the customer PEP to verify the origin and integrity of incoming artifacts. A tightly controlled verification window ensures that older signing keys cannot be used indefinitely, limiting the exposure of older artifact signatures.

VI. Release and Downstream Enforcement

33. Actual-Request Binding and Liveness

A valid authorization token functions as a strictly bound presentation artifact. It must be inextricably tied to the specific downstream request that creates the real-world effect.

When the token is used, the downstream enforcement point (PEP) does not simply check if the token signature is valid. It recalculates the execution context dynamically. The PEP hashes the actual HTTP method, the actual route, and the actual consequence digest of the live request, and compares them against the token's bound claims. If an attacker intercepts a token and attempts to replay it against a different target or with a modified payload, the actual-request binding breaks (`assertVerifiedBinding` fails), and the execution is mechanically rejected.

34. Verification Profiles by Risk and Boundary

Not all consequences carry the same execution risk, so the engine dynamically assigns strict verification profiles tailored to the nature of the enforcement boundary.

For low-risk, internal operational moves, a PEP verifying the cryptographic signature of a short-lived release token locally might be sufficient. However, high-risk profiles—such as financial transactions or production data disclosures—mandate aggressive `sender-constrained` presentation and online liveness verification.

Under high-risk profiles, the PEP cannot rely on local signature verification alone. It is forced to execute a synchronous, online liveness check against the central control plane. This architectural requirement ensures that even if a token was validly issued milliseconds ago, it can still be halted. If an authorized control-plane revocation or policy-freeze action marks the authorization inactive, the PEP's online liveness check can reject it before the network boundary is crossed. The Measurement Plane may surface the signal that triggers operator or policy action, but it cannot revoke authorization by itself.

35. Enforcement Boundary Adapters

While the `SignedReleaseAuthorization` token is inherently protocol-agnostic, the actual enforcement of that token must adapt strictly to the specific downstream protocol.

This is handled by Enforcement Boundary Adapters, which reside within the PEP. These adapters mechanically translate the engine's abstract cryptographic verification requirements into the native semantic language of the customer's infrastructure. By treating the adapter as a formal boundary, the system ensures that the `actual-request binding` and replay protections are successfully enforced regardless of the transport layer. Whether the action is transmitted over a synchronous HTTP REST API, published to an asynchronous Kafka queue, written as a database record, or routed through an Envoy service mesh, the boundary adapter ensures the exact same cryptographic gating rules apply.

36. Customer PEP and No-Bypass Enforcement

The abstract decisions of the control plane are only advisory until they meet the physical network boundary known as the Customer Enforcement Point (PEP).

A PEP is only effective when it covers the protected routes and eliminates parallel execution paths. Using the `IntegrationKit` generated during the onboarding phase, the customer runs bypass probes against their own boundary to gather execution evidence.

Until the customer supplies No-Bypass Evidence showing that the boundary successfully blocked unauthorized bypass attempts, the platform treats the customer deployment as incomplete. Without explicit proof that the physical network boundary works as intended, protection is grounded in assumption rather than evidence, keeping the engine in a restricted evaluation posture.

37. Downstream Execution Receipt

Once the PEP executes or rejects the operation, the runtime flow must be formally closed by sending an Execution Receipt back to the central control plane.

This receipt is not a simple boolean flag. It is a structured artifact that strictly binds the observed downstream result back to the original admission digest, the presentation context, the replay nonce consumption, the specific identity of the PEP, timestamps, and digest-safe error references.

Crucially, recording an observed outcome does not grant that outcome any retroactive execution authority. Instead, the receipt functions as an immutable ledger entry. It allows the consequence to safely exit the active enforcement state and transition into downstream operational phases like reconciliation, incident handling, measurement, or data-driven policy review, cleanly separating the execution phase from observability.

38. Unknown Outcomes, Reconciliation and Concurrency

Because unpredictable network partitions and downstream timeouts are inevitable, the platform must operate as a fully stateful, resilient consequence engine.

If a release authorization is issued but a network error prevents the PEP's Execution Receipt from arriving, the system explicitly transitions the consequence into a strictly typed `Unknown Outcome` state. It never naively retries the operation, which could trigger a dangerous duplicate execution. Instead, the system manages safe concurrency through strict `idempotencyKey` bindings, single-use nonce consumption, transactional outbox queuing, and delayed receipt matching.

This robust state machine explicitly handles partial downstream failures, missing receipts, and duplicate webhook deliveries. By mechanically resolving these edge cases, the engine can deterministically dictate whether it is mathematically safe to retry the execution automatically, or whether it must aggressively freeze the pipeline and escalate the unknown state to a human operator for manual reconciliation.

Part III — Domain Adapters and Technical Appendix

VII. Programmable Money and Crypto Consequences

39. Crypto as an Integrated Domain Adapter

Unlike traditional wallet infrastructure that treats crypto as an isolated silo, Attestor treats crypto strictly as a high-consequence domain adapter operating entirely within the primary consequence engine.

It is not a separate product, nor does it maintain a parallel authority system. A crypto transaction is subjected to the exact same rigorous admission state machine as standard fiat `money-movement` or `data-disclosure` actions. It inherits the exact same consequence identity, unified reviewer queue, release authorization token, and cryptographic proof lineage logic. The only difference is the specialized enforcement boundary it ultimately reaches.

40. Admission Before Signing

The foundational architectural rule of the crypto domain is strict `admission before signing`. The engine unequivocally forbids any cryptographic signing or transaction broadcasting from occurring before full policy admission is complete.

When an AI agent intends to execute a crypto action, it must propose a structured intent envelope to the central control plane. The engine comprehensively evaluates this intent against the active policy, checking authority boundaries, executing shadow simulations, and verifying risk scopes.

If—and only if—the admission state machine reaches a terminal `admit` decision, it issues a constrained release authorization. Only when the customer's downstream enforcement boundary successfully verifies this token does the action reach the customer's actual wallet or HSM to produce a cryptographically signed blockchain transaction or `ERC-4337 UserOperation`.

41. Crypto Consequence Grammar

Because blockchain transactions are immutable and inherently high-risk, the crypto domain forces the generic consequence engine into a significantly stricter evaluation grammar, defined internally as the `general-crypto-transaction-gate`.

The system refuses to evaluate raw, opaque hexadecimal bytecodes. Instead, it demands deeply structured crypto intent. The grammar requires mathematically precise bindings for the network `chain ID`, the specific asset reference, the resolved counterparty address, the exact amount, and the target smart contract.

Furthermore, the admission engine strictly enforces the evaluation of the `callData` digest, the `EIP-712` typed data digest, results from mainnet simulation forks, and the exact cryptographic reference of the ultimate wallet, `Safe`, or institutional custody provider executing the action.

42. Crypto Consequence Families

To prevent ambiguous intent from bypassing the ruleset, the strict crypto grammar maps exclusively to a specific, hardcoded set of high-risk action families.

The native supported consequence families explicitly recognized by the admission engine include native gas token transfers, `ERC-20` transfers and approvals, `EIP-2612` permit signing, decentralized swap executions, cross-chain bridge transfers, `Safe` multisig transaction proposals, `ERC-4337 UserOperations`, session-key lifecycle grants, delegated protocol authorizations, and machine-to-machine `x402` payment channels.

Any crypto intent that cannot be explicitly mapped to one of these well-defined, verifiable families is automatically rejected by the parser before policy evaluation even begins.

43. Signing Boundary

The Attestor runtime decides *whether* an exact action may reach signing, but it does not sign transactions itself. The customer's wallet, `Safe`, custody provider, or bundler boundary performs the actual signing and on-chain execution.

The engine serves strictly as the pre-signing admission layer. For crypto consequences, LLM output can never substitute for structured evidence about the chain, spender, permit domain, quorum, `UserOperation` hash, or delegation scope. These must arrive as structured evidence from the relevant customer-controlled integration before admission.

VIII. Proof, Privacy and Failure Handling

44. Decision Lineage

To provide detailed observability, consequence decisions generate a structured lineage showing how the request was evaluated.

This lineage traces the consequence from the initial `Canonical Consequence Envelope` through the Hard Gates phase, the ingestion of contextual signals, relationship mapping, and monotone fusion. It records the final decision, the exact digest of the generated `SignedReleaseAuthorization`, the PEP's verification payload, and the ultimate downstream Execution Receipt.

By capturing this lifecycle, the lineage creates verifiable evidence of how the consequence moved from a raw proposal through evaluation, release presentation, and recorded outcome.

45. Tamper-Evident History

The decision lineage is stored to preserve historical integrity.

Successive evaluation events, approvals, and receipts are recorded into a hash-linked structure. Each event binds the cryptographic hashes of the previous state, securely referencing the `SignedAssurancePackets` and policy bundles that justify the current transition.

This design produces a tamper-evident history ledger. Modification to a policy rule, an approval signature, or an execution receipt breaks the integrity verification. A retroactive alteration of a blocked or permitted action's history would require rewriting the subsequent hash chain.

46. Data Minimization and Safe Remediation Feedback

The control plane strictly regulates what it can reveal to the outside world or back to an AI model (`data-minimization-redaction-policy`).

In the case of a `Review` or `Block` outcome, the system emits only a safe remediation path containing a reason code, missing evidence type, required approval class, and allowed next step. This feedback serves exclusively as a controlled remediation path, strictly isolated from prompt-injection return channels. Elements such as raw prompts, private policy thresholds, raw evidence content, credentials, customer payloads, and downstream error bodies are structurally redacted. A model can never "learn" the secret threshold by probing the failure codes.

47. Failure Modes, Guards and Recovery

The admission engine assumes by default that AI-generated intent may be hostile, compromised, or hallucinated. To defend against this, it employs specific internal guards targeting known failure classes like prompt injection, scope explosion, and tool-result poisoning.

All tool-derived content and model outputs enter the system as strictly untrusted variables. They remain quarantined until they are explicitly bound to cryptographically verified evidence and validated against the active policy bounds.

If an internal guard detects suspicious patterns, hallucinated parameters, or insufficiently trusted tool outputs, it immediately injects blocking pressure into the monotone fusion layer. The core invariant holds: an AI model's output or a raw tool result can never independently authorize its own execution.

48. Assurance Case and Formal Design Model

The system's safety claims are documented in a formal Assurance Case and supported by state modeling.

The Assurance Case explicitly details the evidence supporting the system's core safety claims. It systematically documents the protected invariants, the required evidence, underlying operational assumptions, and potential defeaters.

In parallel, the core admission state machine is modeled using TLA+ specification language. The TLA+ model provides a design-level basis for reasoning about permitted transitions and safety invariants. It does not constitute a formal verification of the full TypeScript runtime or customer deployment.

IX. Outcomes, Measurement and Closed-Loop Governance

49. Outcome and Incident Lifecycle

The lifecycle of a consequence does not end when the release token is issued; it follows a complete, continuously observable path from admission all the way to final downstream effect and reconciliation.

Once an operation is admitted, physically executed by the PEP, and cryptographically receipted, the system continues to monitor its operational wake. If any subsequent downstream issues arise—such as a contested financial transfer, a reversed database record, or an event formally flagged by human operators as a security incident—that telemetry is inextricably tracked back to the original consequence digest.

This leads to a formal, data-driven postmortem. Rather than blindly updating heuristic weights in an opaque AI model, this strict lifecycle process feeds immutable, structured evidence directly into the Policy Foundry for replay, simulation, and heavily governed policy review.

50. Outcome Source Classes

The Measurement Plane ingests a vast array of telemetry, but it operates under a strict data-typing regime to preserve the distinct fidelity of every signal.

The system maintains rigid boundaries between different classes of evidence. It mathematically treats a cryptographic downstream `Execution Receipt` entirely differently from a subjective reviewer label, an operator annotation, a confirmed incident ticket, or an inferred heuristic signal. It explicitly refuses to merge these distinct, multi-fidelity data classes into a single, generic "status" field.

By preserving these source boundaries, the system mechanically prevents lower-trust inferred inputs or hallucinated model feedback from overriding or masquerading as confirmed, cryptographically verified incident state.

51. Assurance Measurement Plane

The Assurance Measurement Plane acts as the continuously operating observability layer for the consequence engine, designed to monitor the system's internal health and detect dangerous policy gaps.

It produces bounded, structured evidence regarding model drift, unanticipated policy gaps, heavy review queue load, operational regression, and degraded network conditions. However, to maintain the fundamental security architecture, the Measurement Plane remains structurally and computationally isolated from the active admission state machine.

Its output serves exclusively as an observability signal. While it crucially informs operator interventions, triggers alerts for safety budget pressure, and flags the necessity for new policy candidates in the Foundry, the Measurement Plane holds zero execution authority and can never independently authorize or release a consequence.

52. Safety Budgets and Degraded Modes

The consequence engine operates under the assumption that infinite scalability is a myth in high-security environments. When the Measurement Plane observes high predictive uncertainty, excessive model drift, or a large backlog in the human review queue, it explicitly registers an exhausted safety budget.

In response to this exhaustion, the system automatically shifts active operations into a configured `Degraded Mode`. For high-consequence action classes, this shift mechanically enforces `fail-closed` or `review-required` behavior to safely handle the overflow condition without bypassing rules.

Crucially, heavy operational load, extreme system stress, or an unresponsive downstream AI model can never be used as an excuse to relax the level of control required for execution. When stressed, the system simply falls back to a safer, more restrictive operational state.

53. From Outcome Back to Reviewed Policy

Ultimately, the Attestor platform operates as a continuously observed and strictly human-governed control loop.

All verified outcome data and measurement evidence are systematically fed back into the Policy Foundry. This allows the system to identify enforcement gaps and propose highly targeted, data-backed policy candidates for future deployment.

While this continuous operational evidence is valuable for surfacing candidate rules, the structural separation of the planes guarantees that observational data can never autonomously mutate an active policy, issue a release token, or grant execution rights. Every executed consequence securely informs the governance of the next, driving continuous improvement without ever compromising the engine's authority boundary.

X. Product Operation and Scope

54. Hosted Tenant and Service Plane

The platform cleanly decouples the business logic of multi-tenant SaaS operation from the strict, high-security consequence evaluation runtime.

The Hosted Tenant and Service Plane operates as a completely separate supporting layer. It exclusively handles standard operational concerns: tenant account provisioning, API-key lifecycle management, rate limiting, billing usage, and feature entitlements. By keeping this service plane isolated, the system ensures that a failure, misconfiguration, or downtime in the billing or account management layer cannot accidentally leak execution authority into the core consequence engine.

55. Adoption Path and Shared Responsibility

Deploying a consequence engine is a deliberate progression.

The adoption path for a tenant follows a strict progression: initial integration mapping, safe shadow observation, data-driven policy discovery, human review of simulated counterexamples, scoped live enforcement, and finally, active proof capture.

Crucially, the security model is built on Shared Responsibility. The central decision platform absorbs the immense complexity of multi-layered admission logic, policy governance, and cryptographic artifact generation. However, the customer's downstream enforcement point (PEP) must physically enforce the network boundary. Without both halves of this equation actively communicating and verifying signatures, the protection loop remains incomplete.

56. What Attestor Is — and Is Not

The platform serves entirely as a strict consequence control system. It separates the AI intent from the actual consequence. The critical boundary lies strictly between a proposed action and a real consequence.

Conclusion

The engine does not attempt to make AI output trustworthy by itself.

It makes consequential execution conditional on a continuous chain of structured intent, verified evidence, bounded authority, active policy, cryptographic integrity, downstream enforcement, and recorded outcome.

The result is not a safer prompt. It is a controlled path from proposed action to real consequence.

References and Design Anchors

The following references inform Attestor’s architecture, vocabulary, threat model, assurance approach, enforcement design, and crypto authorization model.

They are engineering and design anchors. They do not imply conformance, certification, completed production deployment, or formal verification of the complete runtime.

[1] National Institute of Standards and Technology. *NIST SP 800-162: Guide to Attribute Based Access Control (ABAC) Definition and Considerations.*

[2] OASIS. *eXtensible Access Control Markup Language (XACML) Version 3.0 Core Specification.*

[3] National Institute of Standards and Technology. *NIST SP 800-207: Zero Trust Architecture.*

[4] National Institute of Standards and Technology. *Artificial Intelligence Risk Management Framework (AI RMF 1.0).*

[5] OWASP. *Agentic AI Threats and Mitigations.*

[6] IETF RFC 8785. *JSON Canonicalization Scheme.*

[7] IETF RFC 7662. *OAuth 2.0 Token Introspection.*

[8] IETF RFC 7009. *OAuth 2.0 Token Revocation.*

[9] IETF RFC 9449. *OAuth 2.0 Demonstrating Proof of Possession (DPoP).*

[10] IETF RFC 9421. *HTTP Message Signatures.*

[11] SPIFFE and SPIRE. *Workload Identity and Workload Attestation.*

[12] Lamport, Leslie. *Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers.*

[13] Object Management Group. *Structured Assurance Case Metamodel (SACM), Version 2.3.*

[14] GSN Community. *Goal Structuring Notation Community Standard, Version 3.*

[15] W3C. *PROV Data Model (PROV-DM).*

[16] OpenTelemetry. *Logs Data Model.*

[17] EIP-712. *Typed Structured Data Hashing and Signing.*

[18] ERC-1271. *Standard Signature Validation Method for Contracts.*

[19] ERC-4337. *Account Abstraction Using Alt Mempool.*

[20] Safe. *Smart Account Guards and Module Guard Documentation.*

[21] x402 Foundation. *x402 Version 2 Protocol Specification.*

[22] Attestor Research Provenance Ledger. *Repository research, implementation evidence, tests, and documented limitations.*

Technical Appendix: Implementation Traceability

A verifiable mapping of the whitepaper's main concepts to the relevant core packages, architecture documents, and design specifications.

Repository: github.com/AI-gateway-systems/attestor

Component	Main Files and Specifications
Proof reference contract	src/consequence-admission/contracts.ts
Canonicalization and digests	src/release-kernel/release-canonicalization.ts
Signed assurance packet	src/consequence-admission/signed-assurance-packet.ts
Tamper-evident history	src/consequence-admission/tamper-evident-history.ts
Policy bundles and signatures	src/release-policy-control-plane/bundle-format.ts, src/release-policy-control-plane/bundle-signing.ts
Policy resolution and activation	src/release-policy-control-plane/scoping.ts, src/release-policy-control-plane/resolver.ts, src/release-policy-control-plane/activation-records.ts, src/release-policy-control-plane/activation-approvals.ts
Protected release authorization	src/consequence-admission/generic-protected-release-token.ts, src/release-kernel/release-token.ts
Presentation and replay binding	src/consequence-admission/presentation-binding.ts, src/consequence-admission/presentation-replay-ledger.ts
Customer gate / PEP	src/consequence-admission/customer-gate.ts, src/release-enforcement-plane/offline-verifier.ts, src/release-enforcement-plane/online-verifier.ts
Crypto consequence gate	src/consequence-admission/general-crypto-transaction-gate.ts, src/consequence-admission/crypto.ts
Crypto architecture narrative	docs/02-architecture/general-crypto-transaction-gate.md
Receipt integrity	src/consequence-admission/downstream-execution-receipt.ts
External signer trust boundary	src/service/bootstrap/release-tenant-signer-boundary.ts, docs/02-architecture/external-signer-contract-closure.md
Signing provider adapter	src/service/bootstrap/gcp-kms-release-signer.ts, docs/02-architecture/gcp-kms-release-signer-adapter.md
Decision-context drift	src/consequence-admission/decision-context-drift-binding.ts
Data-minimization boundary	src/consequence-admission/data-minimization-redaction-policy.ts

Component	Main Files and Specifications
Crypto domain model	`src/consequence-admission/general-crypto-transaction-gate.ts`, `docs/02-architecture/general-crypto-transaction-gate.md`